
FAST National University of Computer & Emerging Sciences

Department of Computer Science

PROJECT REPORT

Design and Development of an Autonomous Black Line Following Robot

Group Number 22

Group Members 25M_0608 Muhammad Rafay
25M_0585 Muhammad Fahad Javed

Course DLD Lab

Instructor Ma'am Fozia Solat

Semester Spring 2026

Submission Date 3 May

Submitted in partial fulfilment of the course requirements

1. Introduction

1.1 Background and Motivation

Autonomous mobile robots have steadily moved from the domain of science fiction into practical everyday engineering applications. Industries such as manufacturing, warehousing, logistics, and healthcare now routinely deploy robots that navigate their environments without any human intervention. Among the many types of autonomous ground robots, the **line follower robot** occupies a special place in both academic study and industrial practice. It is a robot designed to detect and continuously follow a predefined path, most commonly a black line drawn on a white surface, by reading sensor signals and adjusting its movement in real time.

1.2 Importance of the Project

At a practical level, the skills developed during this project are directly transferable to more advanced work. Sensor interfacing, PWM-based motor speed control, and modular code design are all skills used in professional embedded systems development. Line follower robots also feature prominently in national and international robotics competitions, including RoboCup Junior. Students who have built and debugged a working robot of this type are significantly better prepared for such competitions and for further coursework in robotics, mechatronics, and control systems

3. Methodology / Working Principle

3.1 IR Sensor Working Principle

The entire navigation capability of this robot rests on a simple but reliable optical detection principle. Each IR sensor module contains two components mounted side by side inside a single housing: an **infrared LED** that emits a constant beam of infrared light downward toward the surface, and a **photodiode** that measures the intensity of the infrared light reflected back from that surface.

The key physical property being exploited is the difference in **infrared reflectivity** between black and white surfaces. A white surface reflects the majority of incoming infrared light back toward the sensor. When the photodiode receives a strong reflected signal, the onboard comparator circuit outputs a digital **HIGH (logic 1)** signal. A black surface, in contrast, absorbs most of the infrared

light. Very little is reflected back, the photodiode receives minimal signal, and the comparator outputs a digital **LOW (logic 0)**.

Each sensor module includes a small potentiometer that adjusts the comparator's switching threshold. Calibrating this correctly before each use is important, as ambient lighting conditions affect the background infrared level in the room. Proper calibration ensures that the sensor reliably distinguishes between the black tape and the white surface even under varying indoor lighting.

3.2 Decision-Making Logic

Two sensors are mounted side by side beneath the front of the robot chassis, spaced approximately 2 cm apart and positioned 6 to 8 mm above the ground surface. With this arrangement, four distinct combinations of sensor readings are possible, each corresponding to a specific positional relationship between the robot and the line:

4. Components and Software Used

Every component used in this project was selected for availability, ease of integration with the Arduino ecosystem, and suitability for a beginner-to-intermediate level embedded systems project. The table below lists all hardware components and the software tool used, along with the quantity and specific role of each:

Component	Qty	Role in Project
Arduino Uno R3	1	Central microcontroller; reads sensors, runs decision logic, drives outputs
IR Sensor Module (TCRT5000)	2	Detect black/white surface contrast via infrared light reflection
L298N Dual Motor Driver	1	H-bridge IC; controls direction and speed of two DC motors independently
DC Gear Motors (6V)	2	Drive the rear wheels; speed and direction set by L298N signals
Robot Chassis + Wheels	1 set	Physical frame, rear drive wheels, front caster for support
9V / 12V Battery Pack	1	Powers motors via L298N; Arduino powered separately via USB or
Jumper Wires (M-M, M-F)	As needed	All electrical signal and power connections
Arduino IDE 2.x	Software	Code editor, compiler, uploader; built-in Serial Monitor for debug

Table 2: Complete Components and Software List

5. System Design

The hardware system can be understood as three subsystems connected to the Arduino Uno: the two IR sensor modules that feed digital signals into the Arduino's input pins, the L298N motor driver that receives control signals from the Arduino's output pins and delivers amplified power to the motors, and the power supply that provides appropriate voltages to both subsystems. The complete pin-by-pin wiring is described in the table below:

Signal / Wire	Arduino Pin	Description
IR Left — OUT	A0	Left sensor digital output (LOW=black, HIGH=white)
IR Right — OUT	A1	Right sensor digital output
Both IR — VCC	5V	Sensor power supply
Both IR — GND	GND	Sensor ground reference
L298N — ENA	Pin 5 (PWM)	Left motor speed control (PWM duty cycle)
L298N — IN1	Pin 6	Left motor direction A
L298N — IN2	Pin 7	Left motor direction B
L298N — IN3	Pin 8	Right motor direction A
L298N — IN4	Pin 9	Right motor direction B
L298N — ENB	Pin 10 (PWM)	Right motor speed control (PWM duty cycle)
L298N — 12V	Battery (+)	Motor power input (9-12V, separate from Arduino logic)
L298N — GND	Battery (-) / GND	Common ground — connects all three circuits

Table 3: Complete Pin Connection Reference

6. Implementation

6.1 Hardware Assembly

Physical construction began with the robot chassis. The two DC gear motors were mounted at the rear of the chassis frame using the supplied metal brackets and M3 screws. Rubber-tyred wheels were pressed onto the motor output shafts. A small ball-caster wheel was bolted to the front underside of the chassis, providing a stable three-point support so the robot could rotate freely

without tipping. This differential-drive configuration means the robot turns by running its two rear motors at different speeds, rather than needing a steerable front wheel.

The Arduino Uno was mounted on the upper surface of the chassis using 10mm nylon standoffs, which keep it elevated above the chassis metal to prevent accidental short circuits. The L298N motor driver was similarly elevated and placed near the centre of the chassis for convenient access to its screw terminals.

6.2 Circuit Connections

All electrical connections were made using jumper wires – male-to-male for connecting the motor driver terminal blocks and male-to-female for connecting to Arduino header pins. Each wire was routed carefully and folded against the chassis to prevent it from hanging loose and catching on the wheels. Both sensor VCC and GND were connected to the Arduino 5V and GND pins respectively. The sensor output wires connected to A0 and A1.

6.3 Code and Programming

The program was written and uploaded using the Arduino IDE. Development was structured in stages. The first stage was a sensor test sketch that simply read both IR sensor values and printed them to the Serial Monitor every 200 ms. This confirmed that the wiring was correct and that the sensors produced LOW over black and HIGH over white as expected. The second stage added motor control functions, each tested individually through temporary Serial command handling before the main decision logic was added. The final stage assembled the complete program and tuned the speed values through on-track testing.

Code :

```
const int IN1=3;
const int IN2=4;
const int IN3=5;
const int IN4=6;
int left,leftMost,center,right,rightMost;
void setup() {
  pinMode(A0,INPUT);
  pinMode(A1,INPUT);
  pinMode(A2,INPUT);
  pinMode(A3,INPUT);
  pinMode(A4,INPUT);
  pinMode(IN1,OUTPUT);
  pinMode(IN2,OUTPUT);
  pinMode(IN3,OUTPUT);
  pinMode(IN4,OUTPUT);
  Serial.begin(9600);
}

void loop() {
  left=digitalRead(A1);
  right=digitalRead(A3);
  center=digitalRead(A2);
  leftMost=digitalRead(A0);
  rightMost=digitalRead(A4);

  if(center==0&&left==1&&right==1) moveForward();
  else if(left==0)
  {
    turnLeft();
  }
  else if(right==0)
  {
    turnRight();
  }
  else if(leftMost==0)
  {
    turnLeft();
  }
  else if(rightMost==0)
  {
```

```

        turnRight();
    }
    else if(center==0&&left==0&&leftMost==0&&rightMost==0&&right==0) stopMotors();
    else stopMotors();
}

void moveForward()
{
    digitalWrite(IN1,LOW);
    digitalWrite(IN2,HIGH);
    digitalWrite(IN3,HIGH);
    digitalWrite(IN4,LOW);

}

void stopMotors()
{
    digitalWrite(IN1,LOW);
    digitalWrite(IN2,LOW);
    digitalWrite(IN3,LOW);
    digitalWrite(IN4,LOW);
}

void turnLeft()
{
    digitalWrite(IN1,LOW);
    digitalWrite(IN2,HIGH);
    digitalWrite(IN3,LOW);
    digitalWrite(IN4,HIGH);
}

void turnRight()
{
    digitalWrite(IN1,HIGH);
    digitalWrite(IN2,LOW);
    digitalWrite(IN3,HIGH);
    digitalWrite(IN4,LOW);
}

```

} Code Explanation (Paragraph Form)

This program controls a line-following robot using five IR sensors and a motor driver. Four pins (IN1–IN4) are used to control the direction of two motors, while five sensors connected to A0–A4 detect the position of the black line. A value of 0 represents black, and 1 represents white.

In the setup() function, sensor pins are set as inputs and motor pins as outputs. The loop() function continuously reads all sensor values and makes decisions based on the line position. If only the center sensor detects the line, the robot moves forward. If the line is detected on the left or leftmost sensor, the robot turns left. Similarly, if the line is on the right side, it turns right. If all sensors detect black or no valid condition is met, the robot stops.

Movement functions control the motors by setting pins HIGH or LOW. Forward motion runs both motors ahead, while turning is achieved by rotating motors in opposite directions. This continuous process allows the robot to follow the line smoothly and adjust its path in real time.

7. Results and Output

The robot was tested on a closed oval track measuring approximately 120 cm in total path length, constructed from 2 cm wide black insulation tape applied to a smooth white cardboard surface. The track included two straight sections and two semi-circular bends. Ten consecutive runs were recorded, with the robot started from the same position each time and timed using a stopwatch.

Performance Metric	Measured Outcome
Overall track completion rate	8 / 10 runs completed autonomously (80%)
Average lap time (120 cm oval)	18 to 22 seconds per lap
Straight-line drift (max observed)	Within ± 3 to 4 mm of centre
Oscillation at full speed (PWM 255)	Noticeable zigzag; eliminated at PWM 180
Sensor response lag	Sub-millisecond; no perceptible delay during testing
Success rate on sharp bends (<6 cm r)	65%; improved further with sensor gap tuned to 2 cm

Table 4: Measured Performance Results

Performance Analysis

The 80 percent success rate was considered a good result for a first working prototype. Both failures occurred at the same location: the inner corner of the tighter semi-circular bend, which had a radius of approximately 5 to 6 cm. Analysis of the failure suggested that the robot was approaching this corner too fast for the sensor spacing to provide correction in time. The inner sensor lost the line before the correction turned the robot back onto track, causing it to run straight off the bend.

8. Challenge Faced

Sensor Height Calibration

Getting the IR sensors to the correct height above the ground surface took significantly more time than expected. When mounted too high, roughly 15 mm or above, both sensors detected white

regardless of whether the black line was directly beneath them, because the reflected beam spread out too wide and returned enough ambient signal to trigger the HIGH output. When mounted below 3 mm, even minor surface irregularities such as slight warping in the cardboard or small particles of dust caused random signal noise. The solution was to fabricate an adjustable mounting bracket from a strip of aluminium sheet metal, bent at right angles and drilled with slotted holes, so the sensor height could be set precisely and locked without needing to remove and reattach the sensors each time. The optimal range of 6 to 8 mm was found to produce clean, noise-free signals.

Future Improvements

The most significant improvement that could be made to this robot is the implementation of a **PID (Proportional-Integral-Derivative) controller** to replace the current binary logic. In a PID line follower, the robot reads its lateral error from the line continuously and adjusts the motor speed difference proportionally to that error. This eliminates oscillation entirely and allows the robot to follow sharp curves smoothly at higher speeds. PID controllers are widely used in industrial motion control and represent the natural next step in learning embedded control systems.

— End of Report —